Chapter 12 worked solutions

Efficient Execution

This guide covers Exercises 1–4 and 10–11. It follows the timing of a generated response, counts the memory occupied by attention scores, and works through the online softmax update in two tile orders. The companion Jupyter notebook covers Exercises 5–9 and is also available as a PDF.

1. Draw a response timeline

Imagine asking a decoder to finish a four-token prompt. During **prefill**, the decoder processes all four prompt tokens and builds their key–value cache. The last prompt position produces a row of logits, meaning one score per vocabulary token. That row supplies the distribution for selecting the first output token. The first token is therefore selected when prefill finishes, 80 milliseconds after the model starts.

Write the five output tokens as $y_1, ldots, y_5$. The first decoding pass takes y_1 as its input, appends its key and value, and uses the resulting logits to select y_2. Each later pass feeds back the token that was just selected. The exercise ignores the time spent sampling.

Work just completed	Model input for that work	Token selected	Elapsed time
Prefill	All four prompt tokens	y_1	80 ms
Decoding pass 1	y_1	y_2	92 ms
Decoding pass 2	y_2	y_3	104 ms
Decoding pass 3	y_3	y_4	116 ms
Decoding pass 4	y_4	y_5	128 ms

Every row after the first adds 12 milliseconds. In general, output token y_i in this example is selected at $80 + (i - 1) \cdot 12$ milliseconds. Prefill supplies one of the five predictions, leaving four decoding passes. The request ends when y_5 is selected. Feeding y_5 back would start work on a sixth prediction.

The **time to first token** is **80 ms**, and the **total model execution time** is $80 + 4 \cdot 12 = 128$ **ms**. Both times start at the beginning of prefill. The total covers the model work specified in the exercise.

2. Compare two batches

The single request from Exercise 1 produces five output tokens in 128 milliseconds. In the batched setup, three requests start together and each produces five tokens. The 100-millisecond prefill supplies the first token for each request. Each subsequent batched pass advances all three requests by one token. Their selection times are 100, 124, 148, 172, and 196 milliseconds, so every request finishes at $100 + 4 \cdot 24 = 196$ milliseconds.

Aggregate output throughput counts all output tokens completed across the active requests and divides by the time for the whole run. The numerator counts generated tokens only, and the denominator includes prefill because the run starts there. Convert milliseconds to seconds to obtain tokens per second.

Setup	Output tokens across the run	Per-request completion time	Aggregate output throughput
One request	$1 \cdot 5 = 5$	128 ms	$5/0.128 = 39.0625$ tokens/s
Three requests in one batch	$3 \cdot 5 = 15$	196 ms	$15/0.196 \approx 76.5306$ tokens/s

The batch produces three times as many tokens in $196/128 = 1.53125$ times the elapsed time, which raises the aggregate rate. Each request in that batch takes 68 milliseconds longer to finish. Dividing 196 by three would give a share of elapsed time per request, while the three users experience their waits concurrently. Each waits the full 196 milliseconds for its complete response.

Per-request completion time answers how long one user waits for the whole response. If that user asks when text starts appearing, use time to first token instead. That is 80 milliseconds in the single-request run and 100 milliseconds in the batched run. Aggregate throughput answers how many output tokens the system delivers per second across requests.

3. Count one intermediate

In one attention head, each of the 2,048 queries has a score against each of the 2,048 keys. Put queries on rows and keys on columns. The full score array contains $2,048 \cdot 2,048 = 4,194,304$ entries per head. There are 16 heads in this one layer, so one sequence needs

$$16 \cdot 2,048^2 = 67,108,864$$

score entries when all the full arrays are stored. Multiply by the bytes per score to obtain the payload. One mebibyte (MiB) is $2^{20} = 1,048,576$ bytes.

Bytes per score	Bytes for one head	Bytes for 16 heads	MiB for 16 heads
2	8,388,608	134,217,728	**128 MiB**
4	16,777,216	268,435,456	**256 MiB**

The shape is the same in both cases. Doubling the bytes per element doubles the storage. A causal mask can block some query–key pairs, but this count concerns the allocated full square arrays. The blocked entries still occupy slots in those arrays.

Two costs outside this score count are **the softmax probability array**, if it is also materialized, and **the key and value storage**. The probabilities contain normalized weights derived from the scores, while the keys and values supply the projected vectors that attention reads. Each has its own shape and dtype. To estimate peak memory, we would also track when the other arrays coexist with the score arrays.

4. Follow a changing maximum

Call the tile with scores $(0, \ln 3)$ and values $(2, 4)$ tile A. Call the tile with score $\ln 6$ and value 1 tile B. Each tile brings another piece of the same query's score row. After processing any nonempty set of permitted scores, the state stores the largest score m, the sum of shifted exponentials ℓ, and the sum of those exponentials times their values u. Here each value is a scalar, so u is also a scalar.

$$m = \max_j s_j, \qquad \ell = \sum_j e^{s_j - m}, \qquad u = \sum_j e^{s_j - m} v_j.$$

Both sums use the same reference score m. When a new tile raises the maximum to m', each old contribution needs a factor of $e^{m - m'}$. This follows from $e^{s_j - m'} = e^{s_j - m} e^{m - m'}$. Apply the factor to both old sums, then add the new tile's terms using the new maximum.

Order A, then B. The first tile sets $m = \ln 3$. Its two shifted exponential weights are $e^{0 - \ln 3} = 1/3$ and $e^{\ln 3 - \ln 3} = 1$. The state is

$$(m, \ell, u) = \left(\ln 3, \frac{1}{3} + 1, \frac{1}{3} \cdot 2 + 1 \cdot 4 \right) = \left(\ln 3, \frac{4}{3}, \frac{14}{3} \right).$$

Tile B raises the maximum to $\ln 6$. Rescale the earlier sums by $e^{\ln 3 - \ln 6} = 1/2$. The new score equals the new maximum, so its shifted exponential is one.

$$\ell' = \frac{1}{2} \cdot \frac{4}{3} + 1 = \frac{5}{3}, \qquad u' = \frac{1}{2} \cdot \frac{14}{3} + 1 \cdot 1 = \frac{10}{3}.$$

The final state is $(\ln 6, 5/3, 10/3)$ and the output is $(10/3)/(5/3) = 2$.

Order B, then A. Tile B now sets $m = \ln 6$ from the start. Its single score has shifted exponential one, so the first state is $(\ln 6, 1, 1)$. When tile A arrives, its largest score $\ln 3$ is lower than $\ln 6$. The maximum stays at $\ln 6$, and the old sums receive a factor of $e^{\ln 6 - \ln 6} = 1$. The incoming scores are measured relative to $\ln 6$, giving $1/6$ and $1/2$.

$$\ell' = 1 + \frac{1}{6} + \frac{1}{2} = \frac{5}{3}, \qquad u' = 1 + \frac{1}{6} \cdot 2 + \frac{1}{2} \cdot 4 = \frac{10}{3}.$$

Tile order	State after the first tile (m, ℓ, u)	Factor on old sums at the second tile	State after the second tile (m, ℓ, u)
A, then B	$(\ln 3, 4/3, 14/3)$	$1/2$	$(\ln 6, 5/3, 10/3)$
B, then A	$(\ln 6, 1, 1)$	1	$(\ln 6, 5/3, 10/3)$

The first tile in either order starts from empty sums. The meaningful difference in rescaling occurs on the second update. A followed by B changes the maximum, while B followed by A already has the largest score.

For **direct softmax**, exponentiate the original three scores to get $(1, 3, 6)$, whose sum is 10. The attention output is the weighted sum using the normalized weights $(1/10, 3/10, 6/10)$.

$$O = \frac{1 \cdot 2 + 3 \cdot 4 + 6 \cdot 1}{10} = \frac{20}{10} = 2.$$

The online sums use the shifted weights $(1/6, 3/6, 6/6)$. Both the direct numerator and denominator are divided by six, which preserves their ratio. Whichever tile order we use, the final sums describe the same three entries with the same maximum. The equality is exact for the arithmetic above. A floating-point implementation can show small differences from summing in different orders.

10. Plan a fair kernel comparison (optional)

Suppose a decoder must process an entire prompt before it can return the first new token. During this **prefill** phase, every prompt position supplies a query. Compare two implementations of causal attention on the projected query, key, and value tensors from that phase. The materializing implementation writes the score and probability arrays to device memory. The fused implementation processes score tiles and carries forward the softmax statistics it needs. In both cases, query position i may read keys 0 through i, including its own key.

Use prompt lengths of 512, 2,048, and 4,096. Fix the batch size at two sequences, with eight heads and a head width of 64. Use FP16 inputs and outputs and FP32 softmax accumulation in both implementations. State the intermediate formats too, since a full score array stored in FP32 occupies four bytes per entry. Use the same GPU, tensor layout, head grouping, causal scaling and mask, and disable dropout. Prepare one seeded batch of projected arrays for each length and use the exact same arrays in both paths. Record which kernels actually run. An automatically fused reference would change the comparison.

Start by checking correctness on a short prefix. Compare each output head and query row with a reference that materializes scores in float64 and subtracts the row maximum before exponentiating. This also checks that each row uses only the keys it should see. For the three timed lengths, compare the two outputs on the same inputs before collecting times. For example, for seeded, unit-scale inputs in this setup, compare the fused output with the timed materializing path and require each output coordinate to agree within an absolute tolerance of 2×10^{-3} plus 2×10^{-3} times the magnitude of the reference coordinate. Record the maximum absolute difference and investigate any failure before interpreting timing. Tiny differences can occur when a tile changes the order in which the sums are evaluated. A different mask changes the computation itself.

Measure the attention operation within prefill. Load the projected inputs onto the device before the timer starts and stop when the attention output is ready on the device. Both paths include the score calculation, causal masking, softmax and weighted value sum. The reference includes the operations that write and reread its full intermediates. The fused path includes all of its tile processing and any workspace it uses. If an implementation constructs a mask or workspace as part of every call, include that work. The Q, K and V projections, the output projection and token sampling are outside this attention-only interval. A separate full-model prefill measurement could include those stages to show how much the attention improvement changes time to first token.

Warm up each implementation at each shape, then run 30 paired trials with the order alternating. Synchronize the device before starting and before stopping a wall-clock timer, so any host work inside the attention call is counted. Report median milliseconds and the middle half of the trials for each length. In a separate run, reset the peak allocated-memory statistic once the inputs are resident. Record the peak above that baseline. Use an allocation trace to distinguish temporary workspace from the common output buffer at each point in time. Record allocator-reserved memory separately if reporting it, since reserved blocks can stay on the device after tensors have been released.

To test the data-movement explanation, collect hardware counters in separate, warmed profiling runs on a GPU that exposes them. Count bytes read from and written to high-bandwidth memory by all kernels in one attention call. Compare the sum for the same input and prompt length. A smaller peak workspace tells us that fewer bytes are stored at once. A smaller traffic count tells us that fewer bytes crossed the device-memory interface during the operation. A reduction in both elapsed time and transferred bytes would support the proposed explanation. Timing alone cannot separate memory traffic from changes in scheduling or arithmetic.

11. Explain a changing bottleneck (optional)

During prefill on a prompt of length n, one head computes scores for n queries against n keys. Its grid is $n \times n$. The causal mask allows the diagonal and everything below it. The sketch uses positions numbered from zero and assumes $n \geq 4$.

Prefill query / key	0	1	…	$n-2$	$n-1$
0	✓	—	…	—	—
1	✓	✓	…	—	—
…	…	…	…	…	…
$n-2$	✓	✓	…	✓	—
$n-1$	✓	✓	…	✓	✓

The first decoding pass after prefill consumes the token selected from the prompt's last logit row. That token has position n. Its layer computes one new query and appends one new key and value to the n already cached. The new query can see every key from 0 through n, including the key from its own input. The score grid is therefore $1 \times (n+1)$ per head.

Cached query / key	0	1	…	$n-1$	n
n	✓	✓	…	✓	✓

A materialized prefill score array has n^2 slots even though only $n(n+1)/2$ pairs are allowed. A fused kernel can avoid writing the full array of scores and the full array of probabilities between its operations. At $n = 2,048$, the square grid has 4,194,304 slots per head. The first cached step has only 2,049 scores per head. It already has little score storage to eliminate. It still needs the cached key and value vectors for the visible positions, and the full model still runs projections and feed-forward layers for the new token. At batch size one, reading those arrays or dispatching many small kernels can account for a larger share of the total time. The shapes explain why the opportunity changes. Which cost dominates is a measurement question.

Measure the time spent in attention, projections and the feed-forward network with a kernel trace for prefill and for the first cached step. Keep the same model, input, batch size and precision while switching attention implementations. Then measure the attention kernels' device-memory read and write traffic for those phases. For example, a large drop in prefill writes with little change in decode reads would fit the explanation that full intermediates mattered mostly during prefill. If decode attention gets faster but its share of the step is small, the trace will show why the overall decode time moves little.